

RAI TECHNOLOGY UNIVERSITY

(Estd. under Karnataka Act No. 40 of 2013)

COMPUTER ORGANIZATION AND ARCHITECTURE

Assignment No: 1

FOREST COVER PROJECT

Subject Code: OCS354T

Submitted By:

Student Name: NEETU K R

University Enrolment Number: IT101024

Program Name: Bachelor of Technology

Specialization: Information Technology (AIML)

Semester: 5th Sem

Session: 2026–2027

Submitted To:

Faculty Name: Mr Abhishek S

Department of Computer Science & Engineering, College of Engineering

FOREST COVER TYPE PREDICTION

Machine Learning Based Classification System

ABSTRACT

Forest cover classification is an important machine-learning application in which terrain and environmental measurements are used to identify the vegetation cover associated with a geographical location. Manual classification can be difficult when many numerical variables have to be considered simultaneously. This project develops a Forest Cover Type Prediction system that accepts terrain-related inputs and predicts one of the seven forest cover classes in the Covertypes-style classification problem.

The project uses a supervised learning workflow. The training data are loaded using Pandas, feature engineering is performed to create additional terrain information, and the resulting features are used to train classification models. The main implementation includes a Random Forest classifier, while Gradient Boosting and K-Nearest Neighbors can be used for model comparison. The trained model and supporting information are saved using Joblib so that the prediction application can reuse the same feature structure.

Two engineered variables are particularly useful in the implementation. Distance_To_Hydrology is calculated from horizontal and vertical hydrology distances using the Euclidean-distance formula. Terrain_Ruggedness is calculated as Elevation multiplied by Slope. During prediction, the Streamlit frontend accepts important terrain measurements through interactive controls and fills remaining model inputs using training-data median values. The engineered features are then recalculated before prediction.

The final application provides a simple graphical interface rather than requiring the user to execute Python commands manually. It displays the predicted cover class and, where supported by the selected model, class probabilities. The project demonstrates the complete machine-learning lifecycle from data preparation and feature engineering to model training, model persistence, frontend development, testing and result presentation.

Keywords: Forest Cover, Machine Learning, Random Forest, Gradient Boosting, KNN, Feature Engineering, Streamlit, Classification, Covertypes.

1. INTRODUCTION

Forests are complex ecosystems whose distribution is influenced by elevation, slope, aspect, distance from water resources, distance from roads, distance from fire points and other geographic variables. Because these variables interact with one another, automatic classification can assist in organizing large collections of terrain observations into meaningful forest-cover categories.

The Forest Cover Type Prediction project applies supervised machine learning to a multiclass classification problem. Each training record contains numerical cartographic measurements and a target `Cover_Type` value. The target represents a forest cover category, such as Spruce/Fir, Lodgepole Pine, Ponderosa Pine, Cottonwood/Willow, Aspen, Douglas-fir and Krummholz.

The system begins by reading the training dataset and separating the target variable from the input variables. An important part of the implementation is feature engineering. Instead of using only the original columns, the system derives `Distance_To_Hydrology` and `Terrain_Ruggedness`. These variables provide additional information that may help a classifier recognize patterns in terrain.

After preprocessing, the data are divided into training and testing subsets. A Random Forest classifier is trained on the training portion. The model is then saved as a `.pkl` file together with the model column order and median values. This makes the prediction stage consistent with the training stage.

A Streamlit frontend provides the user interface. The user enters terrain information using number controls, presses the prediction button and receives the predicted cover type. This design makes the project suitable for classroom demonstration because the user can change input values and observe how the prediction changes without editing the Python training code.

2. HISTORY / EXISTING TECHNOLOGY / LITERATURE SURVEY

Forest-cover mapping has historically relied on field surveys, aerial photographs, geographic information systems and satellite-based remote sensing. These approaches can provide detailed geographical information, but they may require significant data collection, processing and expert interpretation. Machine learning provides an additional computational approach for classifying observations when labelled training data are available.

Decision-tree methods became popular for classification because they can represent nonlinear relationships between input variables. Ensemble methods extend this idea by combining multiple learners. Random Forest is one such ensemble method: it trains many decision trees using randomized subsets of data and features and combines their predictions. This approach is well suited to tabular classification with nonlinear relationships.

Gradient Boosting is another ensemble approach. It builds models sequentially, with later learners concentrating on errors made by earlier learners. K-Nearest Neighbors follows a different strategy: it predicts a class based on the labels of nearby training observations under a chosen distance measure.

The Covertype dataset is widely used as a machine-learning benchmark for multiclass classification. It contains cartographic variables describing terrain and environmental conditions. The project uses the structure of this problem to demonstrate a practical end-to-end classification application.

Existing technology therefore includes statistical classification, decision trees, ensemble learning, nearest-neighbor methods and remote-sensing based systems. The present project focuses specifically on a tabular machine-learning workflow and an interactive frontend, making the model easier to demonstrate and test.

3. PROBLEM STATEMENT

Forest-cover prediction from multiple terrain measurements is a multiclass classification problem. A user or analyst may have several numerical measurements for a location but may not know the corresponding forest cover category. Manually interpreting many variables at the same time is difficult and may not be consistent.

The problem addressed by this project is to design and implement a machine-learning system that receives numerical terrain information and predicts the corresponding forest cover type. The system should maintain the same feature definitions during training and prediction, handle missing input values where necessary, calculate engineered features consistently and present the result through an understandable frontend.

The application is intended as an academic machine-learning demonstration. It is not intended to replace professional forest inventories, ecological field surveys or official land-management decisions.

Problem Requirements

- Load a labelled forest-cover dataset from CSV format.
- Perform feature engineering using terrain and hydrology measurements.
- Split the dataset into training and testing portions.
- Train a multiclass classification model.
- Save the trained model and supporting preprocessing information.
- Accept user inputs through a graphical frontend.
- Generate a forest-cover prediction using the saved model.
- Display the predicted class in a clear and readable format.

4. OBJECTIVES

- To develop a machine-learning system for forest cover type classification.
- To preprocess the forest-cover dataset and prepare numerical features for supervised learning.
- To create useful engineered features such as Distance_To_Hydrology and Terrain_Ruggedness.
- To divide the data into training and testing sets using a reproducible random state.
- To train classification models such as Random Forest, Gradient Boosting and KNN for experimentation and comparison.
- To save the trained model so that it can be reused without retraining every time the application starts.
- To maintain the same feature-column order during prediction as used during training.
- To build an interactive Streamlit frontend for entering terrain parameters.
- To display the predicted cover type in a user-friendly format.
- To display class probabilities when the selected model supports probability prediction.
- To provide an educational demonstration of the complete machine-learning lifecycle.

4.1 Expected Outcome

The expected outcome is a working application in which a user enters terrain measurements and receives a predicted forest cover class. The project also demonstrates how a trained model can be separated from the user interface by saving it as a reusable model file.

5. SCOPE OF THE PROJECT

prediction. It includes data loading, feature engineering, model training, model persistence, frontend The project scope covers the design and implementation of a numerical-data machine-learning classifier for forest cover type development and basic testing.

5.1 Functional Scope

- CSV-based training data loading.
- Feature engineering from hydrology and terrain variables.
- Training/testing split with stratification.
- Machine-learning classification using supported models.
- Saving and loading trained models using Joblib.
- Interactive user input through Streamlit.
- Prediction and probability display where available.

5.2 Technical Scope

The implementation is based on Python and commonly used data-science libraries. Pandas is used for tabular data handling, NumPy for numerical operations, Scikit-learn for machine learning, Joblib for model persistence and Streamlit for the graphical frontend.

5.3 Limitations

- The application depends on the quality and representativeness of the training dataset.
- The current frontend exposes selected important variables and uses training-data median values for other model columns.
- Predictions are model outputs and should not be interpreted as professional ecological assessments.
- Model performance can vary depending on the selected algorithm, hyperparameters and data distribution.

6. METHODOLOGY

The project follows a supervised machine-learning workflow consisting of data loading, feature engineering, preparation, model training, evaluation, model persistence and frontend prediction.

6.1 Data Loading

The training data are loaded from train.csv using Pandas. The Id column is an identifier and is removed from the feature set. Cover_Type is retained as the target variable.

6.2 Feature Engineering

Distance_To_Hydrology is calculated as the Euclidean distance using horizontal and vertical hydrology distances:

$$\text{Distance_To_Hydrology} = \sqrt{(\text{Horizontal_Distance_To_Hydrology}^2 + \text{Vertical_Distance_To_Hydrology}^2)}$$

Terrain_Ruggedness is calculated using elevation and slope:

$$\text{Terrain_Ruggedness} = \text{Elevation} \times \text{Slope}$$

6.3 Data Splitting

The prepared data are divided into training and testing sets using an 80:20 ratio. Stratification is used so that the class distribution is represented in both subsets. A fixed random_state of 42 makes the split reproducible.

6.4 Model Training

The primary implementation uses RandomForestClassifier with 100 estimators and random_state=42. The project can also include Gradient Boosting and KNN models for experimentation. The selected model is trained using the prepared training features.

6.5 Model Persistence

The trained model is saved as forest_model.pkl. The ordered feature names are saved as model_columns.pkl and training-set median values are saved as median_values.pkl. These files allow the Streamlit application to reproduce the prediction input structure.

7. SOFTWARE & HARDWARE REQUIREMENTS

7.1 Software Requirements

Component	Requirement
Operating System	Windows 10/11 or compatible operating system
Programming Language	Python 3.x
IDE / Editor	Visual Studio Code or equivalent Python IDE
Libraries	Pandas, NumPy, Scikit-learn, Joblib
Frontend Framework	Streamlit
Data Format	CSV
Model Format	Joblib / PKL
Browser	Google Chrome, Microsoft Edge or equivalent

7.2 Hardware Requirements

Component	Recommended Requirement
Processor	Intel/AMD processor or equivalent
RAM	8 GB or more
Storage	At least 2 GB free for project and packages
Display	Standard laptop/desktop display
Internet	Required for initial package installation; not required for local prediction after setup

7.3 Project Files

File	Purpose
train.csv	Training dataset
train_model.py	Data preparation and model training
app.py	Streamlit frontend and prediction logic
forest_model.pkl	Saved trained model
model_columns.pkl	Saved training feature order
median_values.pkl	Saved median values used for unspecified inputs

8. PROJECT SCHEDULE

The following schedule provides a practical sequence for developing the project from problem identification to final demonstration.

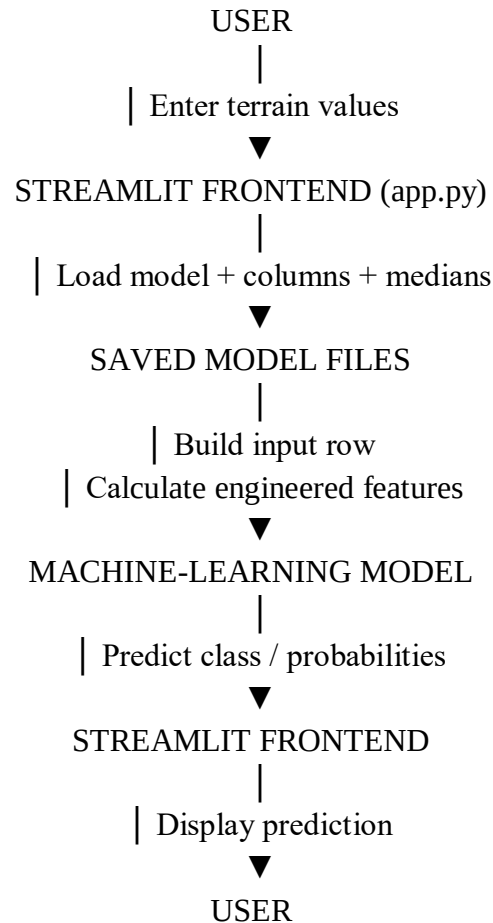
Week / Phase	Activity	Output
Phase 1	Problem identification and literature survey	Problem definition and background
Phase 2	Dataset collection and inspection	Clean understanding of columns and target
Phase 3	Feature engineering and preprocessing	Prepared training features
Phase 4	Model training	Trained classification models
Phase 5	Model evaluation	Test-set performance measurements
Phase 6	Model saving and integration	PKL model/support files
Phase 7	Streamlit frontend development	Interactive prediction interface
Phase 8	Testing and debugging	Validated application
Phase 9	Report and presentation preparation	Final documentation and PPT

8.1 Development Sequence

1. Prepare and verify the dataset.
2. Create engineered features.
3. Split the dataset into training and testing data.
4. Train the selected classification models.
5. Evaluate the models on the test set.
6. Save the model and supporting feature information.
7. Create the Streamlit frontend.
8. Load the saved files inside app.py.
9. Collect user input and recreate engineered features.
10. Run prediction and display the result.
11. Perform test cases and prepare the final report.

9. SEQUENCE DIAGRAM

The following sequence describes the interaction among the user, Streamlit frontend, saved model files and machine-learning model.



9.1 Sequence Explanation

12. The user opens the Streamlit application.
13. The application loads the saved trained model and supporting files.
14. The user enters elevation, slope, aspect, hydrology distances, roadway distance and fire-point distance.
15. The application starts with the training median values for the remaining model features.
16. The application overwrites the fields entered by the user.
17. Distance_To_Hydrology and Terrain_Ruggedness are recalculated using the current input.
18. The input row is arranged in the exact model column order.
19. The saved model performs the prediction.
20. The frontend displays the predicted cover type and probability information where supported.

10. IMPLEMENTATION

10.1 Training Program

The training program loads train.csv, creates the two engineered features, separates Id and Cover_Type, performs an 80:20 stratified split and trains the Random Forest classifier. The model, feature columns and median values are then stored for reuse by the application.

The essential implementation logic is:

```
df['Distance_To_Hydrology'] = np.sqrt(
    df['Horizontal_Distance_To_Hydrology']**2 +
    df['Vertical_Distance_To_Hydrology']**2
)

df['Terrain_Ruggedness'] = df['Elevation'] * df['Slope']

X = df.drop(['Id', 'Cover_Type'], axis=1)
y = df['Cover_Type']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

rf_model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
rf_model.fit(X_train, y_train)

joblib.dump(rf_model, 'forest_model.pkl')
joblib.dump(X_train.columns.tolist(), 'model_columns.pkl')
joblib.dump(X_train.median(), 'median_values.pkl')
```

10.2 Streamlit Frontend

The frontend loads the saved model and support files. Number-input controls are used for the terrain variables exposed to the user. When the prediction button is pressed, a row is created from the saved median values, user inputs replace selected values, engineered features are recalculated and the model produces the final class.

10.3 Cover Type Labels

Class	Cover Type
1	Spruce/Fir
2	Lodgepole Pine
3	Ponderosa Pine
4	Cottonwood/Willow
5	Aspen
6	Douglas-fir
7	Krummholz

11. TEST CASES

Testing verifies that the application accepts valid inputs, handles the prediction flow correctly and displays an understandable result.

TC ID	Test Condition	Input / Action	Expected Result
TC-01	Application startup	Run Streamlit app	Frontend loads without a prediction-time error when required files are present.
TC-02	Default values	Use default terrain values	Prediction button produces a cover-type result.
TC-03	Change elevation	Increase/decrease elevation	Application accepts the new value and produces a prediction.
TC-04	Change slope	Change slope value	Terrain_Ruggedness is recalculated before prediction.
TC-05	Hydrology values	Change horizontal/vertical hydrology distances	Distance_To_Hydrology is recalculated.
TC-06	Roadway/fire values	Change roadway/fire distances	New values are included in the input row.
TC-07	Feature order	Submit a prediction	Input dataframe follows model_columns order.
TC-08	Probability output	Use a model supporting predict_proba	Class probability values are displayed.
TC-09	Repeated prediction	Change inputs and click Predict again	A new prediction is generated without retraining.
TC-10	Missing model file	Temporarily remove a PKL file	Application reports a file-loading error; restoring the file resolves it.

11.1 Testing Notes

The test cases should be executed during the final demonstration on the same environment in which the project is submitted. Numerical accuracy values should be taken directly from the actual training run rather than manually estimated.

12. RESULT AND DISCUSSION

The implemented system provides an end-to-end forest cover prediction workflow. The training stage produces a reusable machine-learning model, while the Streamlit application provides a graphical interface for prediction. The separation between training and inference means that the model does not need to be retrained each time the application is opened.

The Random Forest implementation uses 100 decision trees. Because the model is trained on the prepared feature set, the frontend must reproduce the same engineered variables and column order. The use of saved median values provides a consistent way to populate model features that are not directly exposed through the simplified interface.

12.1 Result Presentation

- Predicted Cover Type: displayed as a class number and corresponding forest-cover name.
- Prediction Confidence: class probabilities can be displayed for models that implement `predict_proba`.
- Interactive Input: users can modify terrain variables and repeat predictions.
- Reusable Model: the trained model is loaded from `forest_model.pkl` rather than retrained in `app.py`.

12.2 Performance Reporting

The training script prints the Random Forest test-set accuracy using `rf_model.score(X_test, y_test)`. The exact value should be copied from the final successful execution into the presentation and report if a numerical performance result is required. This avoids reporting an accuracy value that has not been verified against the project's actual `train.csv` file and execution environment.

12.3 Discussion

The project demonstrates that a machine-learning model can be integrated with a lightweight frontend to make classification accessible to non-programming users. Feature engineering adds derived terrain information, while model persistence separates computational training from interactive prediction. For future work, the system can be extended with stronger hyperparameter tuning, model comparison dashboards, input validation, visual terrain summaries and deployment on a suitable hosting platform.

13. CONCLUSION

The Forest Cover Type Prediction project demonstrates the development of a complete supervised machine-learning application for multiclass forest-cover classification. The workflow begins with a labelled tabular dataset and continues through feature engineering, training/testing split, model training and model persistence.

The project also demonstrates practical software integration. The saved model, feature-column list and median values are reused by the Streamlit frontend so that the prediction stage follows the same feature structure used during training. The frontend allows users to enter terrain information and receive a predicted forest-cover class without directly interacting with Python code.

The application is suitable for academic demonstration of machine learning, feature engineering, model persistence and frontend integration. Its results depend on the quality of the dataset and the selected model, so predictions should be treated as computational classifications rather than professional ecological conclusions.

13.1 Future Enhancements

- Add automated comparison of Random Forest, Gradient Boosting and KNN performance.
- Add cross-validation and hyperparameter tuning.
- Add input validation and clearer error messages.
- Display feature-importance information for tree-based models.
- Add interactive charts for probability distributions.
- Add a dataset-upload option for authorized experimentation.
- Deploy the application to a suitable cloud platform for remote access.
- Add model-version tracking and reproducible experiment records.

14. REFERENCES

1. UCI Machine Learning Repository, Covertypes Dataset, a benchmark dataset for forest-cover classification.
2. Scikit-learn documentation, RandomForestClassifier and supervised learning algorithms.
3. Scikit-learn documentation, GradientBoostingClassifier and KNeighborsClassifier.
4. Pandas documentation, DataFrame operations and CSV data handling.
5. NumPy documentation, numerical operations and mathematical functions.
6. Joblib documentation, model persistence and serialization utilities.
7. Streamlit documentation, application development and interactive widgets.
8. Python Software Foundation, Python 3 documentation.

14.1 Project File Reference

File	Role in Project
train.csv	Input dataset used for supervised training.
train_model.py	Creates engineered features, trains the model and saves supporting files.
app.py	Streamlit frontend for interactive prediction.
forest_model.pkl	Serialized trained model.
model_columns.pkl	Feature-column order used during training.
median_values.pkl	Training median values used to populate unspecified model features.